

What was built and how to launch it

A practical guide to the voice-risk detector, protected-action workflow, local setup, classroom demonstration, tests, and current limitations.

Real pretrained audio inference

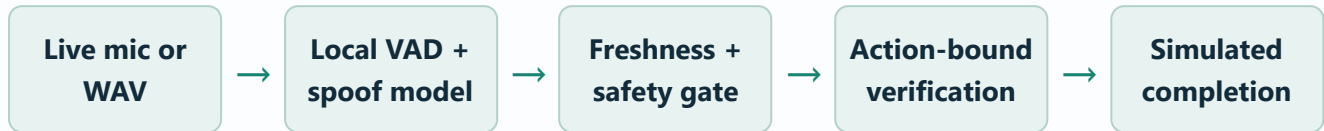
Simulated transfer

Local Docker deployment

Current-build guide for the SIH prototype · 25 September 2026

What VigilVoice does

VigilVoice checks live or recorded speech for usable evidence and synthetic-speech signals before a sensitive action is allowed to continue. The voice result never approves a transfer by itself. A separate verifier must review the exact recipient and amount and provide a one-time code.



What is real

- Silero voice activity detection
- Wav2Vec2 XLSR spoof-classifier inference
- ONNX Runtime CPU execution
- Quality gates and risk policy
- Authenticated sessions and database records
- OTP/action binding, expiry and audit trail

What is simulated

- The bank/payment transfer
- OTP delivery through the local verifier inbox
- The verifier role on one local machine

What was not done

- No model was trained by our team
- No production accuracy claim
- No real SMS, bank or identity integration

The team contribution: connecting real model inference to a controlled action workflow that fails safely, verifies exact details, consumes approvals once, and records evidence.

How the parts work together

1. Browser interface

Native HTML, CSS and JavaScript. Live mode streams authenticated, sequenced 200 ms mono PCM16 frames. WAV upload or a microphone recording up to 10 seconds remains the record-then-analyze fallback.

2. FastAPI backend

Creates private sessions, validates input, runs audio inference away from the event loop, applies the policy, owns actions and exposes the verifier/audit endpoints.

3. Audio pipeline

Silero finds speech and supports duration/quality evidence. Wav2Vec2 outputs a synthetic-speech score. Smoothing slows recovery while a high raw score escalates immediately.

4. Policy and action gate

LOW or ELEVATED evidence may request verification. HIGH, poor, insufficient, stale, invalid or unavailable evidence prevents completion. An optional metadata-only Jev shadow result cannot approve an action.

5. PostgreSQL

Stores sessions, risk evaluations, immutable action details, verification challenges, hashed approval tokens and chronological audit events.

6. Local verifier

A separate page requires a verifier key and shows the exact recipient, amount and single-use code. It demonstrates the intended boundary but is not real independent delivery.

Important source locations

Area	Files
Audio inference	<code>app/services/audio_pipeline.py</code> , <code>vad_worker.py</code> , <code>spoof_detector.py</code>
Protected actions	<code>app/api/v1/actions.py</code> , <code>app/services/action_gate.py</code>
Browser UI	<code>app/static/index.html</code> , <code>app.js</code> , <code>microphone.js</code>
Database	<code>init.sql</code> , <code>config/migrations/001_protected_actions.sql</code>

What happens to each recording

- 1 Validate the file.** Uploads must be uncompressed, mono, 16 kHz, 16-bit PCM WAV, no more than 30 seconds. Browser recordings are capped at 10 seconds.
- 2 Check evidence quality.** The system measures speech duration, clipping, loudness and an estimated signal-to-noise ratio. It does not invent a safe score when evidence is unusable.
- 3 Run spoof inference.** The local ONNX classifier returns a score used by fixed policy thresholds. The score is a model signal, not a probability that a person is fraudulent.
- 4 Assign a risk state.**

State	Meaning	Action behavior
LOW	No strong synthetic signal under current settings	Can request separate verification
ELEVATED	Some spoof signal detected	Can request verification with a warning
HIGH	Strong spoof signal	Transfer is blocked
POOR QUALITY	Clipped, quiet or degraded audio	Ask for better evidence; block
INSUFFICIENT	Too little usable speech	Ask for longer speech; block
UNAVAILABLE	Missing/stale evidence or model failure	Fail closed; block

Current thresholds are exploratory. They are kept configurable for later calibration, but the running profile is deliberately labelled uncalibrated.

Start the prototype on Windows

Prerequisites

- Docker Desktop is installed and running.
- Python is available for the setup helper.
- Chrome or Edge is recommended for microphone recording.
- The first model download needs internet. Normal inference is local afterward.

1 Open PowerShell in the project folder.

```
cd "C:\path\to\SIH"
```

2 Download the pinned models and demo WAVs if the `models` folder is empty.

```
python scripts/fetch_models.py --samples
```

Approximately 358 MB. Existing correct files are checksum-verified and reused.

3 Create local demo secrets and database settings.

```
python scripts/configure_demo.py
```

This creates the ignored `.env` file without printing secrets.

4 Build and start the backend and database.

```
docker compose up -d --build
```

5 Confirm both containers and dependencies are ready.

```
docker compose ps  
Invoke-RestMethod http://127.0.0.1:8000/readyz
```

You should see `vigilvoice_backend`, a healthy `vigilvoice_db`, and `"ready":true` with database, VAD, detector and local verifier available. `/healthz` checks liveness only.

Open the two local screens

Operator dashboard

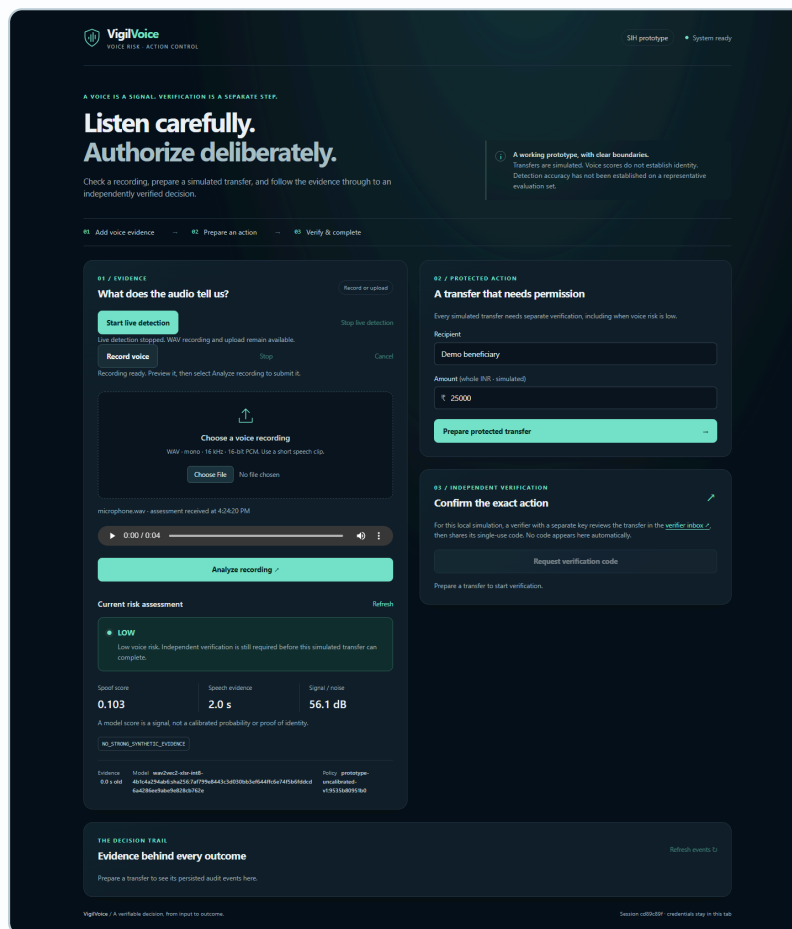
`http://127.0.0.1:8000/`

Start live detection or analyze a WAV, inspect risk, prepare the simulated transfer, enter a code and review the audit.

Verifier inbox

`http://127.0.0.1:8000/verify`

Enter the private verifier key and action ID, review the exact details, then share the one-time code.



Automated fake-device screenshot of the current build. It shows live-stream controls and WAV fallback, but does not prove physical microphone permissions or real-device timing.

Keep the verifier key private. It is stored as `DEMO_VERIFIER_KEY` in the local `.env`. Do not project or commit that file.

Run the complete workflow

1. At the dashboard, click **Start live detection** and speak clearly for several seconds. Show the actual updating risk, including any HIGH or quality rejection. Click **Stop live detection**; old live evidence then becomes unavailable.
2. For the stable action-flow fallback, choose `models/demo/genuine.wav` and click **Analyze recording**. You may also record 3–10 seconds with the microphone, then analyze its WAV.
3. Read the risk, spoof score, speech duration, signal/noise estimate and reason codes. LOW and ELEVATED still require verification.
4. Enter a recipient and whole-INR amount. Click **Prepare protected transfer**. The details become fixed to the generated Action ID.
5. Click **Request verification code**. Open the verifier page in another tab.
6. Enter the verifier key from `.env` and paste the Action ID. Check the recipient and amount before sharing the six-digit code.
7. Enter the code on the dashboard and click **Complete simulated transfer**. The backend rechecks that audio evidence is still safe and fresh, then consumes the approval once.
8. Review the audit events: action created, challenge issued, OTP verified and action completed.
9. Choose **New transfer**, analyze `models/demo/synthetic.wav`, and show a block *if* the current result is HIGH. Report a miss honestly if it is not.

Time limits

- File evidence: 120 seconds
- Live stream evidence: 5 seconds
- OTP: 60 seconds
- Approval: 60 seconds
- Wrong OTP attempts: maximum 3

Presenting alone

After requesting a code, run:

```
python scripts/demo_inbox.py <action-id>
```

This avoids displaying the long verifier key. It still represents simulated local delivery.

The existing 30-second video at `docs/vigilvoice-teacher-walkthrough.mp4` is silent and predates live streaming; label it historical, not current-build proof. The seven-slide PDF has been refreshed, but it is not a narrated backup video.

What was tested

22 / 22

Isolated API check groups passed

Session isolation, stale/HIGH evidence, action-bound OTP, one-time completion, audit rollback, stream loss, capacity and unavailable models. A paused database returned a bounded 503 in the exercised verification path.

10 min

Single-stream stability checkpoint

2,999 sequenced public-audio frames over 600.2 seconds with no growing response backlog. This was not a physical-microphone or multi-user trial.

- ✓ Python: 34 passed, 1 skipped, plus 26 subtests on September 25.
- ✓ UI and microphone Node smoke checks passed.
- ✓ Local VAD/detector available with recorded model hashes.

Exploratory 20-clip test partition

Running default HIGH 0.75	Result
HIGH synthetic recall	7 / 9 scorable (77.8%)
Genuine clips falsely blocked as HIGH	1 / 9 scorable (11.1%)
Quality rejections	2 / 20 total
Service unavailable	0 / 20

An offline exploratory threshold of 0.61 blocked all 9 scorable synthetic historical test clips but falsely blocked 2 of 9 scorable genuine clips. It was **not deployed**. That test partition has already been viewed and is not an untouched final test.

Interpretation: 40 public English clips are useful prototype evidence, not proof of real-world accuracy. Indian-language, replay, phone-channel and unseen-generator validation remains pending.

If the prototype does not launch

Problem	Check or fix
Docker command fails	Open Docker Desktop, wait for it to finish starting, then rerun <code>docker compose up -d --build</code> .
Models unavailable	Run <code>python scripts/fetch_models.py --samples</code> , then restart the backend.
Port 8000 unavailable	Check <code>docker compose ps</code> . Stop another local service using the port or stop the old VigilVoice stack.
Backend exits	Read recent logs with <code>docker compose logs --tail 100 backend</code> .
Database is unhealthy	Read <code>docker compose logs --tail 100 db</code> . Do not delete its volume while preserving data matters.
Microphone button disabled	Use Chrome/Edge on <code>localhost</code> , allow microphone permission, or use a bundled WAV.
Audio rejected	Use 16 kHz mono 16-bit PCM WAV. Record clear uninterrupted speech or use the known demo files.
Database timeout / 503	After readiness recovers, refresh the known action ID and audit before retrying. A timeout does not prove rollback; if no action ID was returned, pause rather than claiming it was not created.
Evidence/code expired	Analyze again and create a new transfer. Expired or locked challenges cannot be reset.

Useful commands

```
docker compose ps
docker compose logs --tail 100 backend
docker compose restart backend
python -m pytest -q
node tests/ui_smoke.cjs
node tests/microphone_smoke.cjs
```

Stop the application

```
docker compose down
```

This stops the containers and preserves PostgreSQL data in its Docker volume. The current application is local-only and binds its ports to `127.0.0.1`.

Is this an MVP?

Working local prototype, not final presentation acceptance. Real inference, continuous streaming, WAV fallback, deterministic blocking, action-bound verification and audit have been exercised. The detector has not passed representative four-language validation, and the final source/offline rehearsal is not frozen.

Working in the local demo

- Docker launch and readiness
- Live and WAV microphone paths
- Pretrained model inference
- Fail-closed action gate
- Simulated protected transfer
- Local verification and audit
- Optional Jev shadow fallback

Still required

- Consented four-language/channel corpus
- Validated threshold or explicit deferral
- Untouched final test
- Separate-verifier and offline rehearsal
- Current narrated backup video
- Final source/package freeze
- Trusted delivery before real use

Recommended next order

1. Obtain consented, source-linked target-domain audio or defer detector replacement.
2. Evaluate with speaker/source-separated validation, freeze the policy and run one untouched test.
3. Rehearse the current microphone/WAV path with a separate verifier and no internet.
4. Only then consider fine-tuning, real verification delivery and a wider pilot.

Presentation wording: “We built a working guarded prototype using pretrained models and measured it on a small public set.” Do not say the team trained the detector or that its real-world accuracy is established.

Current local references: `docs/teacher-demo-guide.md` , `docs/final-development-plan.md` , `docs/validation/stage0-2026-09-24.md` and the V02/V03 receipts.